

Building and Analyzing Monolingual Transformer Language Models in Two Indian Languages

Hindi (Model H) and Nepali (Model L)

Param Modi | Roll No. 2025201087

Language Models and Agents (CL3410)

1 Overview

I built two independent decoder-only Transformer language models from scratch. Model H is trained on Hindi, the higher-resource language, and Model L on Nepali, the lower-resource one. Nothing is shared between them: not a document, not a tokenizer, not a vocabulary entry, not a weight. The work spans three phases. First I collected the corpora and trained the tokenizers, then I implemented and pretrained the models, and finally I finetuned both on a synthetic reasoning task.

Everything that could be held constant was: architecture, parameter count, token budget, hyperparameters and random seed. Only the corpus differs, which is what lets Section 5 attribute the gap between them to the data.

	Model H (Hindi)	Model L (Nepali)
Training tokens	479,808,984	487,001,673
Manual share of training tokens	13.46%	7.98%
Tokenizer fertility (tokens/word)	1.4675	1.5881
Trainable parameters	24,285,184	24,285,184
Test perplexity	26.36	36.79
Bits per byte	0.5448	0.5018
chrF++ (best decoding setting)	19.90	18.35
Reasoning — seen patterns	83.4%	83.7%
Reasoning — unseen entity names	75.0%	62.0%

Table 1: Headline results across all three phases.

2 Phase 1 — Data Collection and Tokenizer Construction

2.1 Language choice

I picked Hindi as the higher-resource language because it has more public web text than any other Indian language, and Nepali from the allowed list for the lower-resource side, mainly because it uses the same Devanagari script. That turned out to matter: with both models reading the same alphabet, the writing system drops out as a variable and whatever separates them has to come from how much text there was and how good it was.

2.2 Sources and manual collection

Public text came from five sources: CC-100, Sangraha in both its verified and unverified forms, mC4, and MADLAD-400. IndicCorp v2 returned zero bytes for both languages, so I dropped it.

The brief asks for at least 20% of training tokens to come from manual collection, allowing OCR, scraping or transcription. I wrote a crawler: it expands a site’s sitemap or MediaWiki API into a resumable SQLite queue, respects `robots.txt`, rate-limits itself, and pulls article text from the HTML. Each document keeps `source`, `url` and `fetched_at`, so any sentence traces back to its page.

2.3 Cleaning pipeline

Both corpora go through the same eight stages, with a checkpoint written after each one so a crash never costs more than a single stage: Unicode normalisation, fastText language identification, boilerplate line removal, a quality filter (Devanagari character ratio, sentence terminator ratio, repeated line fraction), exact deduplication, and MinHash/LSH near-duplicate removal.

Two of those matter more than they look. Hindi and Nepali share a script and much vocabulary, so a Nepali page can land in a Hindi corpus without looking wrong. And syndicated news runs on many sites with only the headline changed; if those survive, the model sees test content during training and perplexity comes out flattered. The downloaded corpora also get an n -gram perplexity filter, which the crawled side does not need.

2.4 Splits

Splits are 80/10/10 at the document level. Rather than shuffle, I assign each document using a hash of its own text:

$$u = \text{blake2b}(\text{"split"}, \text{seed}, \text{text}) / 2^{64}, \quad \text{train if } u < 0.8, \text{ val if } u < 0.9, \text{ else test.}$$

This buys three things. The split reproduces on any machine with no stored index. A duplicated document always hashes to the same place, so a copy can never sit in train while the original sits in test. And hashing manual and downloaded documents as separate streams keeps the manual share nearly equal across all three splits: 12.62%, 12.68% and 12.27% for Hindi.

2.5 Tokenizer

I trained a SentencePiece BPE tokenizer from scratch per language, on that language’s training split only. Three vocabulary sizes (5,000, 8,000 and 10,000) were scored on held-out text by fertility and unknown rate. Both languages ended up at 10,000, the lowest-fertility candidate of the three that still left the embedding table affordable within the 25M budget.

	Model H (Hindi)	Model L (Nepali)
Corpus (<i>all quantities in tokens</i>)		
Training tokens	479,808,984	487,001,673
from manual collection	64,596,209 (13.46%)	38,849,727 (7.98%)
from public corpora	415,212,775 (86.54%)	448,151,946 (92.02%)
Validation tokens	60,226,406	60,899,013
Test tokens	59,778,539	60,373,211
Public corpus downsampled to	37.32%	53.75%
Tokenizer		
Vocabulary size	10,000	10,000
Unknown tokens	5.3 per million	5.1 per million
Top-1000 token coverage	72.52%	62.67%
Fertility (tokens per word)	1.4675	1.5881
Characters per token	3.47	3.95
Bytes per token	8.39	10.24

Table 2: Phase 1 results. Both corpora reach the $\sim 500\text{M}$ training-token target with a near-zero unknown rate. The two bolded rows matter later: a Nepali token covers 22% more bytes than a Hindi one, which is why perplexity and bits per byte disagree in Section 3.3, and it splits unfamiliar words into more pieces, which is the failure diagnosed in Section 4.4.

3 Phase 2 — Model Implementation, Pretraining and Evaluation

3.1 Architecture and parameter budget

Both models use 6 layers with $d_{\text{model}} = 512$, 8 attention heads ($d_k = 64$), a context length of 512 and the output projection tied to the token embedding. The feed-forward sublayer is $\text{Linear}(512 \rightarrow 2048) \rightarrow \text{GELU} \rightarrow \text{Linear}(2048 \rightarrow 512)$, a $4\times$ inner width. Dropout is 0.1 in four places: the summed embeddings, the attention probabilities, and the output of each sublayer before it re-enters the residual stream. The attention-probability dropout is applied after the weights are captured for analysis, so the recorded rows still sum to 1.

Component	Formula	Parameters
Token embedding	$10,000 \times 512$	5,120,000
Positional embedding	512×512	262,144
Per block — W_Q, W_K, W_V, W_O	4×512^2 (no bias)	1,048,576
Per block — feed-forward	$(512 \cdot 2048 + 2048) + (2048 \cdot 512 + 512)$	2,099,712
Per block — 2 LayerNorms	$2 \times (2 \times 512)$	2,048
$\times 6$ blocks	$6 \times 3,150,336$	18,902,016
Final LayerNorm	2×512	1,024
Output head	tied to token embedding	0
Total		24,285,184

Table 3: Parameter budget. The two models land on exactly the same count because both Phase 1 vocabularies came out at 10,000. Weight tying saves 5.12M parameters, 21% of the budget.

The depth/width split follows from the budget. The embedding costs 5.12M before any block exists, leaving about 19M, and a block at $d_{\text{model}} = 512$ costs 3.15M, so six blocks is what fits. The alternatives were roughly

four layers at $d = 640$ or eight at $d = 448$. I took the middle: chaining two premises is a two-step operation and depth is what buys composition steps, whereas width would have spent the budget on richer per-token features a comparison task does not need. Width could not drop much further anyway, since $d = 448$ with 8 heads puts d_k at 56 rather than the familiar 64. The T4 set the ceiling: at 512 context and 32,768 tokens per step, fp16 activations for six blocks fit in 16GB, where a deeper stack would have forced a smaller batch and a noisier gradient.

Multi-head attention, the positional embeddings and the causal mask are written from PyTorch primitives. I deliberately avoided `F.scaled_dot_product_attention`, since a fused kernel hides the attention weights that Phases 2 and 3 go on to analyse.

Each of Q, K, V comes from one 512×512 projection that is then split across heads, not from eight separate projections: $(B, T, d) \xrightarrow{\text{view}} (B, T, h, d_k) \xrightarrow{\text{transpose}(1,2)} (B, h, T, d_k)$. The transpose is the part that matters: it moves h next to the batch dimension, so the scores $(B, h, T, d_k) \times (B, h, d_k, T) \rightarrow (B, h, T, T)$ are $B \cdot h$ independent problems in one matmul and no head mixes into another. Forming the context vectors runs the path backwards, $(B, h, T, d_k) \rightarrow (B, T, h, d_k) \rightarrow (B, T, d)$, which is the head concatenation; it needs `contiguous()` because the transpose left the tensor non-contiguous. Only W_O then mixes across heads. The computation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

The $1/\sqrt{d_k}$ scaling holds the variance of the dot products near 1. Without it the softmax saturates and the gradient vanishes. The mask is applied to the scores before the softmax rather than after, so the surviving weights renormalise to sum to one, and since the diagonal is always allowed, no row is ever fully masked and the softmax cannot produce NaN.

For positions I chose learned absolute embeddings over sinusoidal or RoPE: 262K parameters is affordable, and they reduce the optional “remove positional information” ablation to a single flag. The price is a hard 512-token context limit, which the forward pass raises an explicit error about rather than failing quietly.

I used pre-norm blocks, $x + \text{sublayer}(\text{LN}(x))$, rather than post-norm, $\text{LN}(x + \text{sublayer}(x))$. What differs is the residual path: under pre-norm the skip connection is an unmodified identity from the embeddings to the final LayerNorm, so gradients reach early layers without passing through a normalisation. Under post-norm every residual addition is followed by a LayerNorm, so the gradient is rescaled once per layer on the way back, which compounds with depth and is why such stacks usually need a long warmup to train at all. At 6 layers either would probably work; pre-norm is what let me use a 500-step warmup without the loss spiking early.

The training objective is causal language modelling: the logits at position t are scored against the token at position $t + 1$ by cross-entropy, averaged over every position in the batch, $\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p_\theta(x_{t+1} | x_{\leq t})$. Since p_θ is a softmax over the 10,000-token vocabulary, an untrained model sits at $\ln 10,000 \approx 9.21$, which is where both loss curves start.

Verifying causality. To check the mask works, I changed one token in a sequence and measured how far the change travelled. Every logit before the edited position was bit-identical at 0.00×10^0 , not merely close, while the edited position and everything after it moved. A model ignoring its input would also show zero change before the edit, so the test additionally requires the edited position to move. Two further checks confirm every attention row sums to 1 and every entry above the diagonal is exactly zero.

3.2 Pretraining

Both models were trained with AdamW ($\beta = 0.9, 0.95$), weight decay 0.1 applied to matrices only, a 500-step linear warmup into cosine decay from 6×10^{-4} down to 6×10^{-5} , gradient clipping at 1.0, and fp16 mixed precision on a Kaggle Tesla T4. Each run covered 14,500 steps at 32,768 tokens per step, or 475,136,000 tokens, the same for both. I matched step counts rather than epoch counts so that neither model was trained longer than the other.

Checkpointing. Each checkpoint stores weights, optimizer and scheduler state, step and config, plus the RNG state and data loader position. Those last two are easy to forget and they matter: dropout consumes randomness, so a run resumed without them drifts. Checkpoints are written to a temporary file and renamed, since renaming is atomic and a session killed mid-save must not destroy the only working checkpoint. I tested this by training 40 steps straight through, then repeating it as 20 steps, a save, a reload into a fresh trainer, and 20 more. The largest parameter difference was 0.00×10^0 .

3.3 Language modelling results

That disagreement is the most interesting thing to come out of Phase 2. Perplexity is per token, and the two tokenizers cut text differently: a Nepali token carries 10.24 bytes against Hindi’s 8.39, about 22% more text, so Model L makes a harder prediction at every step. Bits per byte normalises against a unit both models share, $\text{BPB} = (\sum \text{nats} / \ln 2) / \sum \text{UTF-8 bytes}$, which at 3 bytes per Devanagari character is 1.63 bits per character for Hindi and 1.51 for Nepali. What I could conclude at the end of Phase 2 was narrow: the raw perplexity gap overstates the quality gap. Which metric tracks real capability stayed open, and Phase 3 answers it.

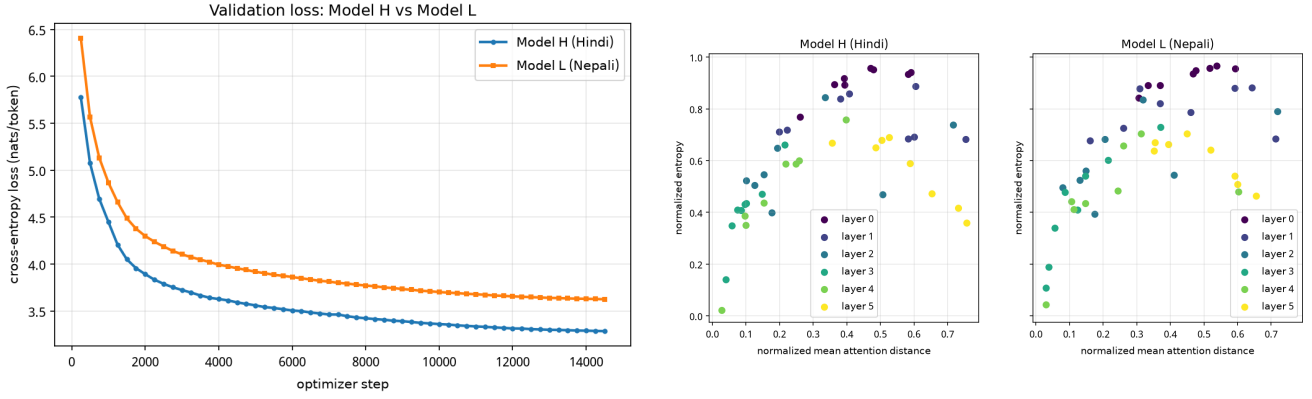


Figure 1: Left: validation loss over pretraining. Both fall from about 9.2 (ln 10000, or uniform guessing) and flatten as the cosine schedule anneals. Right: attention head specialisation, one dot per head, coloured by layer. The two models show the same structure.

	Model H	Model L	
Validation cross-entropy / perplexity	3.2733 / 26.40	3.6114 / 37.02	
Test cross-entropy	3.2719	3.6054	
Test perplexity	26.36	36.79	L is 40% worse
Bits per byte	0.5448	0.5018	L is 8% <i>better</i>
Bytes per token	8.39	10.24	L packs 22% more per token
Unigram baseline perplexity	1,355.2	2,419.0	
Improvement over unigram	51×	66×	

Table 4: Intrinsic evaluation. The two metrics disagree about which model is better.

3.4 Generation quality

For each model I generated 64-token continuations from 300 held-out 64-token prefixes, using greedy decoding and pure temperature sampling at 0.5, 1.0 and 1.5. I used no top- k so that temperature was the only variable.

Setting	Model H (Hindi)					Model L (Nepali)				
	BLEU-4	chrF++	ROUGE-L	Dist-2	rep-4	BLEU-4	chrF++	ROUGE-L	Dist-2	rep-4
greedy	7.02	18.38	0.153	0.247	0.643	2.23	15.43	0.090	0.241	0.656
$t = 0.5$	7.16	19.90	0.157	0.382	0.392	2.35	17.85	0.098	0.448	0.324
$t = 1.0$	4.77	19.16	0.125	0.777	0.037	1.14	18.35	0.077	0.871	0.012
$t = 1.5$	0.99	14.44	0.062	0.961	0.001	0.34	15.72	0.034	0.975	0.000

Table 5: Generation metrics across decoding settings. Quality peaks in the middle; diversity and repetition move monotonically.

Both models trace the usual U-shape. Greedy decoding degenerates badly: a rep-4 of 0.643 means roughly two thirds of the generated 4-grams are repeats. By $t = 1.5$ the repetition has essentially gone, but chrF++ has collapsed too, because the output has stopped making sense.

I report chrF++ rather than BLEU as the primary metric. BLEU measures exact 4-gram overlap against one reference continuation, which for open-ended generation is mostly luck, and it penalises Nepali mechanically, since longer agglutinative words make exact matches rarer whatever the quality. The BLEU gap between the models is 3× and the chrF++ gap is 8%; I trust the second. ROUGE-L I had to implement myself, because the standard `rouge_score` package tokenises with a `[^a-z0-9]+` regex that strips every Devanagari character and quietly returns 0.0.

What the outputs look like. Under greedy decoding the failure mode is a loop. Given a Hindi prefix about bone health, Model H produces हड्डियों को मजबूत रखने के लिए हड्डियों का मजबूत होना जरूरी है. and repeats that clause indefinitely. The grammar is fine and the topic is right, so it has worked out that this is a health article and has nothing further to say. At $t = 1.0$ the loop is gone and Model L writes fluent Nepali energy-sector journalism, with a sensible unit (९० मेगावाट) and a Bikram Sambat date (२०७६ चैत १८). All of it is invented, which is about what 24M parameters should give: correct script, case marking and verb agreement, plausible clauses, no factual grounding.

3.5 Attention analysis

Statistics are averaged over 64 held-out sequences of 256 tokens, giving one row per layer and head, so 48 cells per model. Entropy is normalised by $\ln(t + 1)$ and distance by t , because row t of a causal attention matrix can only spread over $t + 1$ keys and raw entropy would otherwise rise with position for any head at all.

Layer	Model H				Model L			
	Entropy	Distance	Prev-token	Sink	Entropy	Distance	Prev-token	Sink
0	0.907	0.442	0.037	0.046	0.923	0.450	0.032	0.041
1	0.759	0.469	0.037	0.163	0.792	0.438	0.044	0.118
2	0.584	0.289	0.146	0.134	0.603	0.273	0.152	0.096
3	0.413	0.102	0.276	0.040	0.425	0.133	0.298	0.039
4	0.466	0.187	0.249	0.060	0.457	0.227	0.228	0.079
5	0.566	0.575	0.021	0.285	0.604	0.490	0.029	0.176
mean	0.616	0.344	0.128	0.121	0.634	0.335	0.131	0.091

Table 6: Per-layer attention statistics after pretraining.

Three patterns show up in both models. Layer 0 is close to uniform, averaging over the context rather than selecting from it. Layers 3 and 4 are sharpest and most local, with the lowest entropy, shortest distance and highest previous-token rate; some heads are extreme, with Model H’s layer 4 head 0 putting **96.3%** of its attention on the immediately preceding token. Those are the induction-style heads carrying local morphology and agreement, a sensible place for two morphologically rich languages to spend capacity. Layer 5 spreads out again and grows strong sinks, with layer 5 head 5 routing **51.7%** of its attention to position 0. Nothing in the architecture asks for this U-shaped profile across depth; both models arrived at it on their own.

The two differ by under 3% on entropy and distance, so whatever Model L lost to its weaker corpus, it did not end up with a qualitatively different internal organisation. The one real difference is where the sinks form. Model H concentrates them in layer 5 at 0.285, while Model L spreads them earlier.

Model H — attention weights across layers and heads

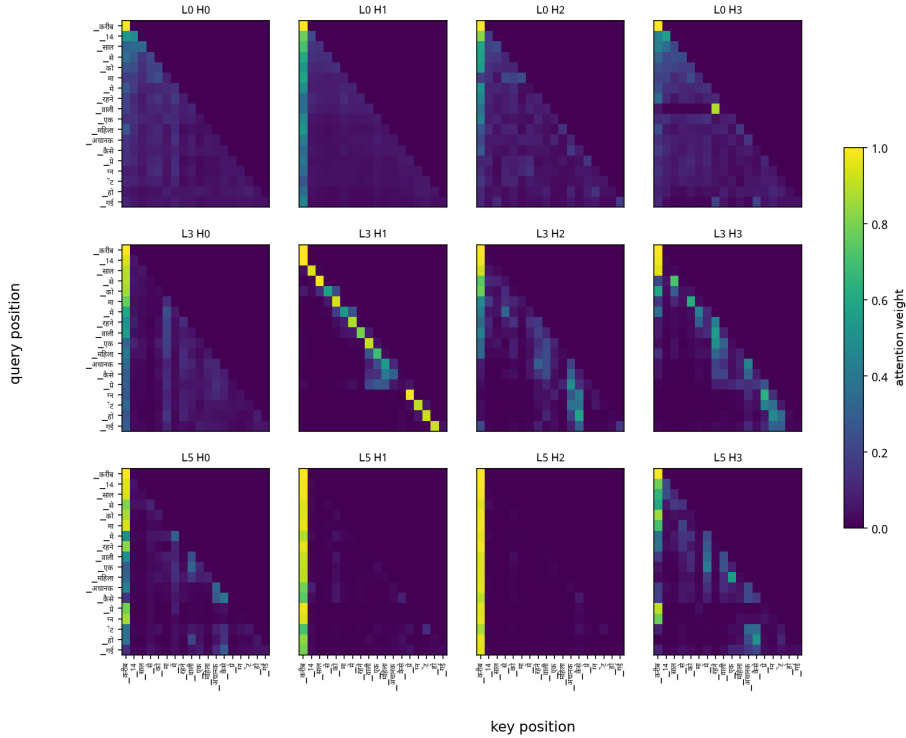


Figure 2: Model H attention weights on one held-out Hindi sentence, for layers 0–3 and 5 across heads 0–3, with Devanagari token labels on both axes. Every panel is lower-triangular, which is the causality proof made visible.

4 Phase 3 — Reasoning Finetuning and Attention Analysis

4.1 The synthetic reasoning dataset

I generated my own comparative reasoning corpus for each language, which means every label is known by construction. Nothing was downloaded. I label the seven question shapes **T0–T6**: each uses the same ingredients, two or three entities compared on one dimension, and they differ only in what is asked.

ID	Premises given	Question asked	Answer	What it tests	In training?
T0	$A > B$	larger? / smaller?	A / B	inverting one relation	yes
T1	$A > B, B > C$	smallest?	C	chaining two inequalities	yes
T2	$A > B, B > C$	largest?	A	chaining two inequalities	yes
T3	$A > B, B > C$	A vs C?	A	transitivity, not a superlative	held out
T4	$A=170, B=165, C=180$	largest?	C	comparing magnitudes	yes
T5	$A=170, B=170$	larger?	neither	recognising a tie	yes
T6	$A > B, B > C$	the middle one?	B	full ordering, not an extreme	held out

Table 7: The seven template families, shown on one running example. T1, T2, T3 and T6 are built from identical premises and differ only in the question, which is what makes the two held-out families a clean test: a model that had learned ordering rather than extremum selection would answer all four.

राम, श्याम से लंबा है। श्याम, मोहन से लंबा है। सबसे लंबा कौन है? (Ram is taller than Shyam; Shyam than Mohan; who is tallest?) → राम
 अनुप निरज भन्दा कान्छो छ र निरज राजु भन्दा कान्छो छ। (Anup is younger than Niraj, Niraj than Raju;
 त्यसैले अनुप र राजु मध्ये कान्छो को हो? so between Anup and Raju, who is younger?) → अनुप

Five comparison dimensions per language, each with its own entity pool: height and age for people, weight and price for objects, speed for vehicles. The pools stay separate because attaching **महंगा** (expensive) to a person’s name is text nobody would write, and the brief asks for natural phrasing rather than templates translated out of English.

Handling the grammar properly doubles as a leakage control. Adjectives agree with their subject (लंबा/लंबी), the question word changes with animacy (कौन for people, कौन सा for objects), and the Hindi possessive agrees with the dimension noun. So every entity in an example must share grammatical gender; otherwise the adjective in the question would give the answer away.

Split	Templates	Entity names	Hindi	Nepali
train	T0, T1, T2, T4, T5	training pool	20,000	20,000
val	T0, T1, T2, T4, T5	training pool	2,000	2,000
test_iid	T0, T1, T2, T4, T5	training pool	1,000	1,000
test_names	T0, T1, T2, T4, T5	held-out pool	1,000	1,000
test_templates	T3, T6	held-out pool	1,000	1,000

Table 8: Three test sets rather than one. Mean encoded length is 27.8 tokens (H) and 26.8 (L), and neither corpus contains a single unknown token.

Why three test sets. One test number cannot tell memorisation apart from real generalisation. The drop from **test_iid** to **test_names** isolates generalisation over entities, and the drop from **test_names** to **test_templates** isolates generalisation over the relation itself. On top of that, every prompt string is blocked from every later split, so the splits are disjoint by construction instead of by filtering afterwards.

Balance controls. Premise order and polarity are both randomised, so a chain appears as “ $A > B, B > C$ ” or “ $C < B, B < A$ ” in either order, and each example is drawn against a round-robin target answer position. The best “always answer the k -th mentioned entity” rule therefore scores only 36.2–36.8% on the in-pattern slices. One structural exception I could not design away: the middle entity of a chain is named in both premises, so it is never mentioned last, and on **test_templates** the positional rule reaches 42.3%. That is the floor those numbers must be read against.

Digit script was measured, not assumed. The numeric templates need numerals, so I counted digit characters across each training corpus first. Hindi uses ASCII digits 96.5% of the time and Nepali uses Devanagari digits 92.0% of the time, so each language’s questions follow its own corpus. Writing २४ into Hindi questions would have made the numeric family fail for a tokenisation reason that has nothing to do with reasoning.

4.2 Finetuning

Each model starts from its own pretrained checkpoint, with its own tokenizer and vocabulary frozen. No token was added and no embedding was resized. Loss is computed on the answer tokens only. For a sequence $[p_0 \dots p_{P-1}, a_0 \dots a_{A-1}, \langle s \rangle]$ the inputs are $\text{ids}[: -1]$ and the targets $\text{ids}[1 :]$, so the first answer token is predicted from index $P - 1$ and targets $[0 : P - 1]$ are masked out. Exactly $A + 1$ positions are supervised. The extra one is the end-of-sequence token, since the model also has to learn where to stop.

Setting	Value	Reason
Initialised from	that language’s pretrained checkpoint	brief: start from own checkpoint
Tokenizer / vocabulary	frozen from Phase 1	brief: unchanged during finetuning
Optimizer	AdamW, $\beta = (0.9, 0.95)$	same as pretraining
Peak learning rate	1×10^{-4}	a sixth of pretraining: adapt, do not overwrite
Schedule	100-step warmup $\rightarrow$ cosine $\rightarrow 1 \times 10^{-5}$	the run is $8\times$ shorter
Optimizer state	fresh, not inherited	old second moments mis-size the first updates
Batch / epochs	32 examples / 3 epochs (1,875 steps)	
Seed	1337	same as pretraining

Table 9: Finetuning hyperparameters, identical for both models.

Validation loss fell from 0.682 to 0.121 for Hindi and 0.623 to 0.102 for Nepali, with answer-token accuracy reaching 94.0% and 94.7%. That accuracy is teacher-forced and measured on data reusing the training question types and names, so it says the optimisation worked and little else.

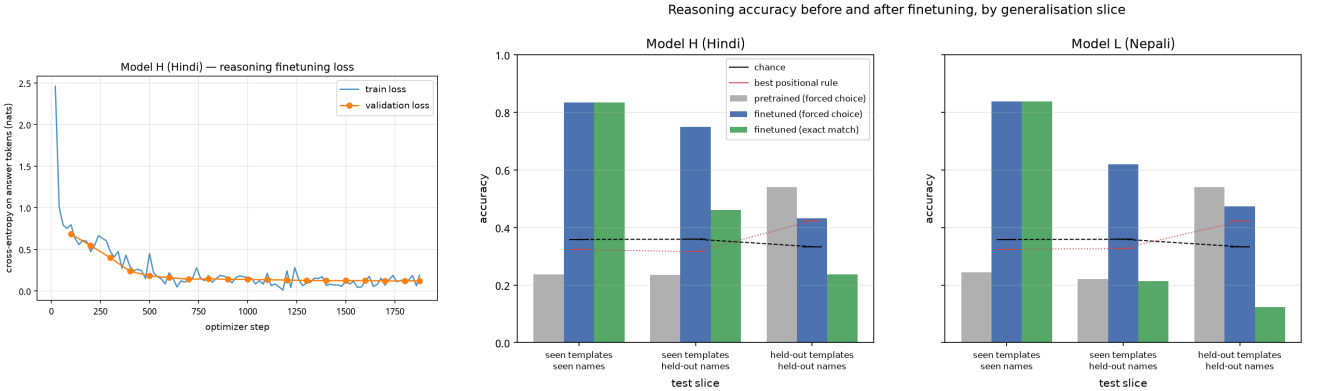


Figure 3: Left: Model H finetuning loss. Right: reasoning accuracy by generalisation slice before and after finetuning, with the chance and best-positional floors marked.

4.3 Reasoning results

I report two scoring protocols. Exact match prompts the model and decodes greedily. Forced choice scores each candidate answer and takes the most likely.

One example shows why both are needed. Asked *अमित, श्याम से बड़ा है। दोनों में बड़ा कौन है?* (*Amit is older than Shyam; who is older?*), the pretrained model answers *दोनों के बीच में एक-दूसरे का:* grammatical Hindi, but not an answer to anything. It is continuing the text, the only thing it has ever been trained to do. Exact match scores that 0.0% on every slice, so reporting “0% to 83%” would credit a gain in output formatting as a gain in reasoning. Forced choice asks the narrower question of whether the model can order the entities at all.

Test slice	Model H (Hindi)			Model L (Nepali)		
	Pre	Post	Exact	Pre	Post	Exact
Seen patterns, seen names	23.7	83.4	83.4	24.5	83.7	83.7
Seen patterns, unseen names	23.6	75.0	46.0	22.1	62.0	21.4
Unseen patterns and names	54.0	43.2	23.7	54.1	47.4	12.4
<i>chance / best positional rule</i>	33–36 / 32–42			33–36 / 32–42		

Table 10: Reasoning accuracy (%). “Pre” and “Post” are forced choice before and after finetuning; “Exact” is exact match after finetuning. Exact match before finetuning is 0.0% on every slice.

Two things in the pretrained rows look wrong at first glance. The 54% on the hardest slice is an artifact: that slice names two of the three entities in the question itself, so a model that simply prefers a recently named

entity is choosing between the right two and gets about 50% for free. And the below-chance scores of 23.7% and 24.5% on the easier slices are not noise. A model guessing at random would sit near chance, so scoring under it means the pretrained model has a systematic preference for the wrong candidate, most likely whichever entity was mentioned most recently.

Template family	H exact	H forced	L exact	L forced
T0 / T1 / T2 / T5, seen names	100.0	100.0	99.6–100.0	99.6–100.0
T4 numeric, seen names	31.7	31.7	30.8	30.8
T0 pair, <i>unseen names</i>	59.4	96.1	24.7	87.7
T1 chain, <i>unseen names</i>	47.7	81.0	12.4	57.8
T2 chain, <i>unseen names</i>	48.5	80.4	18.6	61.5
T5 equality, <i>unseen names</i>	100.0	100.0	100.0	100.0
T3 transitive (held out)	45.6	69.6	17.2	58.9
T6 middle element (held out)	1.8	16.8	7.9	36.7

Table 11: Per-template accuracy (%) after finetuning. Chance is 33%. The aggregate hides three findings that only this breakdown shows.

- (1) **Stated comparison is solved.** The word-based families sit at 100% on familiar names for both languages. The 83% headline is pulled down entirely by one family.
- (2) **Numeric comparison never appeared.** T4 sits at 31.7% and 30.8% against a chance floor of 33%. Both models saw roughly 4,400 numeric training examples and learned nothing from them that transferred. A numeral gets split into subword pieces that carry no information about ordering.
- (3) **Transitivity generalises, but full ordering does not.** T3 never appeared in training and Hindi reaches 69.6% on it against 33% chance, so the model is applying a relation it learned as a superlative to a phrasing it has never seen. T6, though, scores 16.8% for Hindi, which is below chance. The model is not guessing there. It confidently names a largest or a smallest every time, and for that question it is always wrong. What it learned is **extremum selection rather than ordering**.

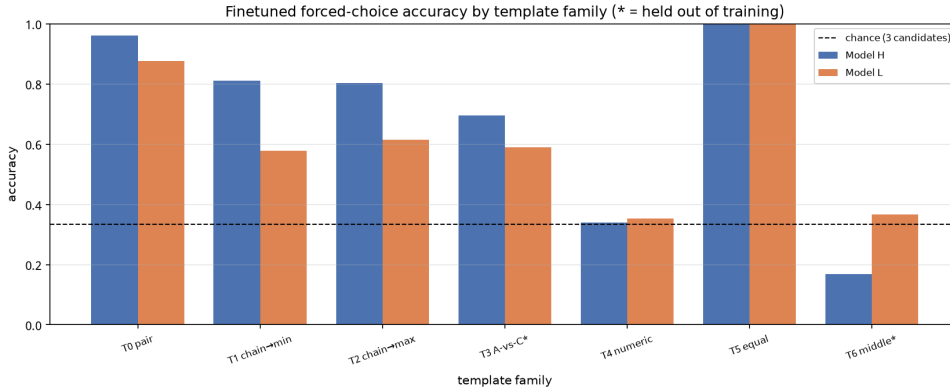


Figure 4: Per-template accuracy for both models, with the chance line drawn. Families marked * were held out of training entirely.

4.4 Qualitative analysis

A representative success on the held-out transitive family, where the premises use one adjective and the question asks for the opposite pole: ज्योति, वंदना से लंबी है और वंदना, किरण से लंबी है। तो किरण और ज्योति में छोटी कौन है? The model answers किरण, correctly.

The most informative failure is the gap between forced choice and exact match on unseen names: 75.0% against 46.0% for Hindi, 62.0% against 21.4% for Nepali. The model ranks the correct unfamiliar name highest, then writes a training name instead. Nepali is much worse, and tokenisation is why: at a fertility of 1.5881 an unfamiliar name splits into more pieces, and every extra piece is another chance to drift. One Nepali output was मता, a fragment that is not a word, so the model lost the thread partway through the name. This is the clearest case in the project of a Phase 1 tokenizer property becoming a Phase 3 capability difference.

The middle-element failures are just as diagnostic. पंकज, मनीष से छोटा है। मनीष, आशीष से छोटा है। बीच में कौन है? has the gold answer मनीष, and the model answers आशीष, naming the largest.

4.5 Attention before and after finetuning

I re-measured attention with the Phase 2 toolkit unchanged, on 64 reasoning prompts neither checkpoint had seen, pushing identical sequences through both models so any difference belongs to the models and not the input.

Statistic	Model H		Model L	
	Pre $\rightarrow$ Post	Δ	Pre $\rightarrow$ Post	Δ
Normalised entropy	0.621 $\rightarrow$ 0.597	-0.024	0.629 $\rightarrow$ 0.647	+0.018
Normalised distance	0.536 $\rightarrow$ 0.453	-0.083	0.531 $\rightarrow$ 0.470	-0.061
Previous-token rate	0.175 $\rightarrow$ 0.169	-0.006	0.182 $\rightarrow$ 0.173	-0.009
Sink rate	0.350 $\rightarrow$ 0.237	-0.113	0.339 $\rightarrow$ 0.227	-0.112
Sink heads (of 48)	24 $\rightarrow$ 12		21 $\rightarrow$ 9	
Content-based heads	22 $\rightarrow$ 30		24 $\rightarrow$ 35	

Table 12: Attention statistics on reasoning prompts, before and after finetuning.

Two changes are large and show up in both models. Attention became more local, with mean distance falling at every layer for Hindi and five of six for Nepali. That is the opposite of what I expected; I had assumed heads would reach further back to compare the entities. The sinks also weakened sharply, with the share of attention dumped on position 0 dropping by about a third and sink heads halving in both models. A sink is what a head does when it has nothing useful to attend to, so on this task many heads found something to do.

The largest single change in each model is the same head type at the same depth: a layer-2 head that turned from a narrow sink into a broader, more local, content-driven head, with sink rate falling 0.547 for Hindi and 0.571 for Nepali. Two independently trained models making the same structural change is the part I find most striking.

These numbers are not comparable to the Phase 2 attention table. Phase 2 measured on 256-token corpus windows and this measures on 30-token reasoning prompts, so only the before-and-after comparison within this section means anything.

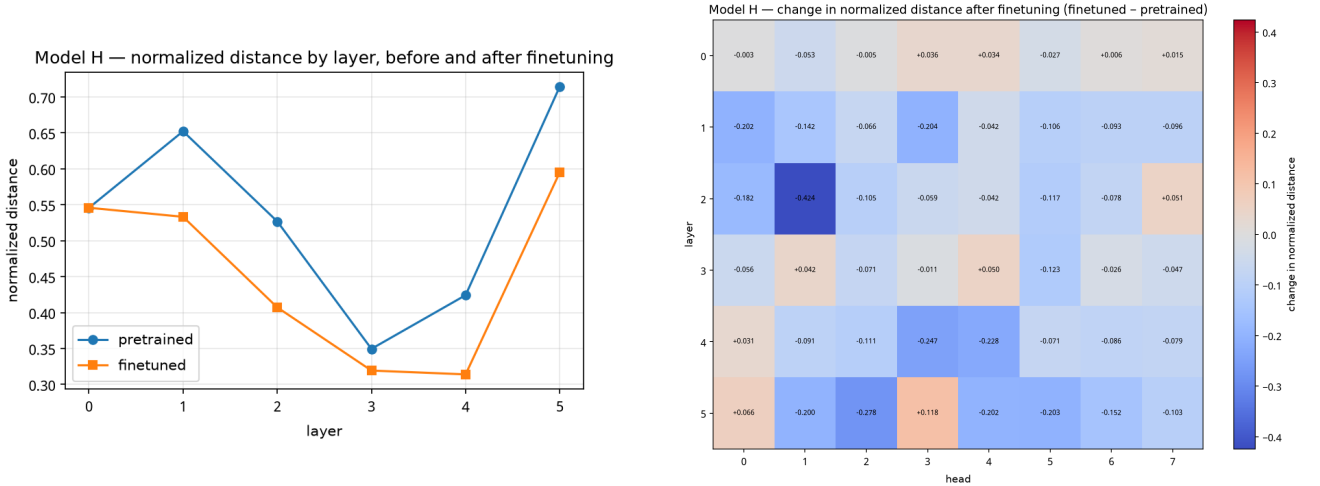


Figure 5: Left: Model H mean attention distance by layer, before and after finetuning; it falls at every layer. Right: the same change per head, on a diverging scale centred at zero. Blue means attention moved closer.

5 Comparing the Two Resource Tiers

How did data scale and quality differ? Nepali is the lower-resource language on every measure, and the shortfalls compound. It had 709M cleaned words against Hindi’s 1,051M, so less aggressive downsampling was possible (53.75% kept against 37.32%), which left a smaller manual share, 7.98% against 13.46%. Its token distribution is flatter too, with top-1000 coverage of 62.67% against 72.52%.

How do language modelling and reasoning results compare? This is the question Phase 2 could not settle. Perplexity says Hindi is better by 40%, bits per byte says Nepali is better by 8%. Reasoning is an independent third measurement, and **it agrees with perplexity**.

The tie on seen patterns is the control that makes the rest interpretable: 83.4% against 83.7% means raw learning capacity is indistinguishable. The gap opens exactly when the task needs generalisation, at 75.0% against 62.0% on unseen names and 46.0% against 21.4% on exact match, where the model must produce an unfamiliar name rather than merely rank it. Nepali’s 4-point lead on the hardest slice is not a win: both sit near the 42% positional floor, and the family driving it is T6, where Hindi falls below chance precisely because it committed harder to the extremum heuristic. So bits per byte measured something real, that Model L compresses its own language slightly better per byte. It did not measure the capability that carries to a downstream task.

Which tokenizer and corpus factors mattered most? Fertility, and I can point at it in the errors. Nepali’s 1.5881 tokens per word against Hindi’s 1.4675 means unfamiliar names split into more pieces, which is the failure in Section 4.4. The flatter token distribution matters too, and is why Model L peaked at $t = 1.0$ rather than $t = 0.5$ in Phase 2. Unknown tokens and vocabulary size I could rule out rather than assume, since both corpora have UNK rates near 5×10^{-6} and both vocabularies are 10,000.

6 Limitations

- Each model was trained once, so none of the comparisons here carry error bars.
- Nepali differs from Hindi in several ways at once, less text, less manual data and a higher-fertility tokenizer, so no single cause can be isolated.
- The reasoning dataset is narrow: seven question types over five comparison dimensions.
- The optional no-positional-embedding ablation was not run.

7 Reflections

Picking the right metric mattered more than improving the model. The most useful result I got was not a number going up. It was finding in Phase 2 that perplexity and bits per byte ranked the models in opposite directions, then designing a third measurement in Phase 3 to break the tie. Reporting only perplexity would have made Model L look much worse; only bits per byte, better. Both readings are defensible from the data and both are incomplete.

Baselines are what turn a number into evidence. 83% means nothing on its own. It became meaningful only once I knew random guessing scores 33%, the best positional shortcut 36%, and a quirk of one question family pushes that to 42%. More than once a result that looked impressive was sitting on a floor. And the one that looked like outright failure, 16.8% on T6, was the most informative number in the project: scoring below chance shows the model learned a specific wrong rule rather than nothing.

Small decisions had large and completely silent consequences. Three stand out. Encoding prompt and answer separately rather than as one string, since the tokenizer can otherwise merge across the boundary and produce a training sequence generation can never reach. Masking the loss at index $P - 1$ rather than P , where being one position out teaches the model to predict the last prompt token and leaves the loss curve looking identical either way. And two entity names where one contained the other, **नेहा** inside **स्नेहा**, which would have corrupted every answer-position statistic in the dataset. None of the three raises an error. I caught them only by writing checks that asserted the property directly.

What I would do differently. I would collect the manual data first and size the download around it, since the 20% requirement was missed largely because the crawl finished last. I would train two seeds per language so the comparisons came with error bars. And I would have put a numeric comparison task into the pipeline far earlier: finding out at the very end that neither model can compare magnitudes was the most surprising result of the project, and deserved more attention than the schedule left.

8 Reproducibility

All code, configs, datasets, logs, evaluation outputs and figures sit in the repository on branch **phase-3**, with per-phase reports under **report/**. Large artifacts are hosted externally with open links in the **README**: checkpoints on Google Drive, tokenized corpora as a public HuggingFace dataset. Every random process is seeded at 1337, the splits are a deterministic hash of the document text, and the reasoning dataset regenerates byte-identically from its seed. The **README** lists the commands to reproduce each phase in order.